

**PENGEMBANGAN SISTEM PELAPORAN MASALAH
LINGKUNGAN DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK
MENDUKUNG *SMART CITY***

Topik: *Application Development dan Intelligence System*

Oleh

Brian Theodore 2602080030

Moh. Khoirul Umam Al Amin 2602198472

Andrea Octaviani 2602075895



**Computer Science
Computer Science Study Program
School of Computer Science
Universitas Bina Nusantara
Bandung
2026**

**PENGEMBANGAN SISTEM PELAPORAN MASALAH
LINGKUNGAN DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK
MENDUKUNG *SMART CITY***

SKRIPSI

Diajukan sebagai salah satu syarat
untuk gelar kesarjanaan pada
Program Studi Teknik Informatika
Jenjang Pendidikan Strata-1

Oleh

Brian Theodore	2602080030
Moh. Khoirul Umam Al Amin	2602198472
Andrea Octaviani	2602075895



**Computer Science
Computer Science Study Program
School of Computer Science
Universitas Bina Nusantara
Bandung**

2026

LEMBAR PERNYATAAN ORISINALITAS

Pernyataan Kesiapan Skripsi untuk Ujian Pendadaran

Kami, Brian Theodore

Moh. Khoirul Umam Al Amin

Andrea Octaviani

Dengan ini menyatakan bahwa Skripsi yang berjudul:

**PENGEMBANGAN SISTEM PELAPORAN MASALAH LINGKUNGAN
DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK MENDUKUNG SMART
CITY
*DEVELOPMENT OF AN ENVIRONMENTAL PROBLEM REPORTING
SYSTEM
INTEGRATING IMAGE AND TEXT DATA TO SUPPORT SMART CITY
INITIATIVES***

adalah benar hasil karya kami dan belum pernah diajukan sebagai karya ilmiah, sebagian atau seluruhnya, atas nama kami atau pihak lain.



Brian Theodore

2602080030



Moh. Khoirul Umam Al Amin

2602198472



Andrea Octaviani

2602075895

Disetujui oleh Pembimbing

Saya setuju Skripsi tersebut layak diajukan untuk Ujian Pendadaran

Bandung, 16 Desember 2025

Budi Juarto, S.T., M.Kom.

D6670

Universitas Bina Nusantara

Computer Science Program
Computer Science Study Program
School of Computer Science
Skripsi Sarjana Teknik Informatika

**DEVELOPMENT OF AN ENVIRONMENTAL PROBLEM REPORTING
SYSTEM
INTEGRATING IMAGE DATA TO SUPPORT SMART CITY INITIATIVES**

Brian Theodore	2602080030
Moh. Khoirul Umam Al Amin	2602198472
Andrea Octaviani	2602075895

ABSTRACT

The slow manual triage process in urban environmental complaint systems—where a single incoming report can take nearly two hours to be routed to the responsible government agency—represents a critical bottleneck in smart city governance. This research proposes an Android application that addresses this bottleneck through on-device AI image classification, eliminating the need for server-side inference and the associated latency, cost, and privacy risks. A comparative benchmark of five modern vision backbone architectures—ConvNeXt V2, EfficientViT, MambaOut, Hiera, and EVA-02—was conducted on a balanced dataset of seven urban environmental issue categories (800 samples per class) sourced from HuggingFace Hub. All models were trained under identical configurations using the AdamW optimizer, cosine annealing scheduling, and automatic mixed precision. MambaOut Tiny

*achieved the highest performance with a test macro F1-score of **0.9917**. The winning model was exported to ONNX format (opset 17) and integrated into a Flutter Android application via ONNX Runtime, achieving numerical parity with the original PyTorch model. The application provides automatic agency routing through a rule-based engine mapping classification results to four operational clusters—corresponding to relevant Indonesian local government agencies (Dinas Lingkungan Hidup, Dinas Bina Marga, Dinas Perhubungan, BPBD)—enabling near-instant report distribution without human intervention. The system is fully offline-capable, with local SQLite storage for report persistence and GPS-based location capture.*

Keywords: *smart city, on-device AI, image classification, ONNX Runtime, Flutter, MambaOut, vision backbone benchmark.*

Universitas Bina Nusantara

**Computer Science Program
Computer Science Study Program
School of Computer Science
Skripsi Sarjana Teknik Informatika**

**PENGEMBANGAN SISTEM PELAPORAN MASALAH LINGKUNGAN
DENGAN
INTEGRASI DATA GAMBAR DAN TEKS UNTUK MENDUKUNG *SMART
CITY***

Brian Theodore	2602080030
Moh. Khoirul Umam Al Amin	2602198472
Andrea Octaviani	2602075895

ABSTRAK

Proses triase manual dalam sistem pelaporan masalah lingkungan perkotaan—di mana satu laporan yang masuk dapat memerlukan waktu hampir dua jam untuk diteruskan ke instansi pemerintah yang bertanggung jawab—merupakan hambatan kritis dalam tata kelola *smart city*. Penelitian ini mengusulkan aplikasi Android yang mengatasi hambatan tersebut melalui klasifikasi citra berbasis AI secara *on-device*, menghilangkan kebutuhan inferensi di sisi server beserta latensi, biaya, dan risiko privasi yang menyertainya. Studi *benchmark* komparatif dilakukan terhadap lima arsitektur *backbone* visual modern—ConvNeXt V2, EfficientViT, MambaOut, Hiera, dan EVA-02—pada dataset seimbang tujuh kategori masalah lingkungan perkotaan (800 sampel per kelas) yang bersumber dari HuggingFace Hub. Semua model dilatih dengan konfigurasi identik menggunakan optimizer AdamW,

cosine annealing scheduling, dan *automatic mixed precision*. MambaOut Tiny mencapai performa tertinggi dengan *test macro F1-score* sebesar **0,9917**. Model terpilih diekspor ke format ONNX (*opset* 17) dan diintegrasikan ke dalam aplikasi Android Flutter melalui *ONNX Runtime*, dengan paritas numerik yang terverifikasi terhadap model PyTorch aslinya. Aplikasi ini menyediakan perutean instansi otomatis melalui mesin berbasis aturan yang memetakan hasil klasifikasi ke empat kluster operasional—sesuai dengan instansi pemerintah daerah Indonesia yang relevan (Dinas Lingkungan Hidup, Dinas Bina Marga, Dinas Perhubungan, BPBD)—memungkinkan distribusi laporan hampir instan tanpa intervensi manusia. Sistem sepenuhnya dapat beroperasi secara *offline* dengan penyimpanan lokal SQLite untuk persistensi laporan dan akuisisi lokasi berbasis GPS.

Kata Kunci: *smart city*, *AI on-device*, klasifikasi citra, *ONNX Runtime*, Flutter, MambaOut, *benchmark backbone* visual.

KATA PENGANTAR

Puji dan syukur kepada Tuhan Yang Maha Esa, atas berkat dan rahmatNya sehingga penulis dapat menyelesaikan laporan Skripsi ini dengan baik dan tepat waktu. Penulis juga tidak lupa berterimakasih kepada pihak yang telah membantu penulis dalam memberikan bimbingan, dukungan, dan doa. Oleh karena itu, pada kesempatan ini penulis ingin mengucapkan terima kasih kepada:

1. Ibu Dr. Nelly, S.Kom., M.M., selaku Rektor Binus University.
2. Bapak Dr. Ir. Derwin Suhartono, S.Kom., MTI, selaku Dekan of School of Computer Science and Head of Computer Science Binus Bandung.
3. Bapak Budi Juarto, S.T., M.Kom., selaku dosen pembimbing yang telah banyak meluangkan waktu untuk memberikan bimbingan, petunjuk, dukungan, dan saran kepada penulis dalam menyelesaikan laporan Skripsi ini.
4. Segenap Dosen Fakultas School of Computer Science yang telah mendidik dan memberikan ilmu selama kuliah dan seluruh staf yang selalu sabar melayani segala administrasi selama proses penelitian ini.
5. Orang tua dan saudara-saudari yang senantiasa mendampingi dengan doa, semangat, dan dukungan materi, sehingga penulis mampu melalui proses pendidikan dan menyelesaikan skripsi ini dengan baik.
6. Semua pihak yang telah membantu dan mendukung penulis selama pembuatan laporan Skripsi ini yang tidak dapat penulis sebutkan satu per satu.

Karena keterbatasan pengetahuan maupun pengalaman penulis, maka penulis yakin masih banyak kekurangan dalam laporan Skripsi ini, Oleh karena itu penulis sangat mengharapkan saran dan kritik yang membangun dari pembaca demi kesempurnaan dokumen skripsi ini.

Bandung, 16 Desember 2025

Penulis

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN SAMPUL	ii
LEMBAR PERNYATAAN ORISINALITAS	iii
ABSTRAK	v
KATA PENGANTAR	ix
DAFTAR ISI	xi
DAFTAR TABEL	xiv
DAFTAR GAMBAR	xv
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Ruang Lingkup	4
1.4 Tujuan dan Manfaat Penelitian	6
1.4.1 Tujuan Penelitian	6
1.4.2 Manfaat Penelitian	6
1.5 Metode Penelitian	7
1.5.1 Pengumpulan dan Persiapan Data	7
1.5.2 Benchmarking Model AI	8
1.5.3 Ekspor Model ke ONNX dan Validasi	8

1.5.4	Pengembangan Aplikasi Android dengan Flutter	8
BAB 2	TINJAUAN PUSTAKA	9
2.1	Landasan Teori	9
2.1.1	<i>Smart City</i> dan <i>Smart Environment</i>	9
2.1.2	Sistem Pelaporan dan Partisipasi Publik	10
2.2	Arsitektur <i>Transformer</i> dan <i>Vision Transformer</i>	10
2.3	Arsitektur <i>Backbone</i> Visual Modern	11
2.3.1	ConvNeXt V2	11
2.3.2	EfficientViT	12
2.3.3	MambaOut	12
2.3.4	Hiera	13
2.3.5	EVA-02	13
2.4	ONNX dan Inferensi <i>On-Device</i>	14
2.4.1	<i>Open Neural Network Exchange</i> (ONNX)	14
2.4.2	<i>ONNX Runtime for Android</i>	14
2.5	Flutter dan Arsitektur Aplikasi <i>Mobile</i>	15
2.5.1	Flutter	15
2.5.2	Riverpod	15
2.5.3	GoRouter	16
2.5.4	SQLite dan sqflite	16
2.6	Metrik Evaluasi Klasifikasi Multi-Kelas	16
2.6.1	<i>Macro F1-Score</i>	16
2.6.2	Metrik Pendukung	17
2.7	Teknik Pelatihan Model	18
2.7.1	Transfer Learning dan Fine-tuning	18
2.7.2	AdamW dan <i>Cosine Annealing</i>	19
2.7.3	<i>Automatic Mixed Precision</i> (AMP)	19
2.7.4	<i>Label Smoothing</i>	19
BAB 3	METODE PENELITIAN	21

3.1	Kerangka Berpikir	21
3.2	Pengembangan Model <i>Artificial Intelligence</i>	21
3.2.1	Dataset	21
3.2.1.1	Sumber Data	21
3.2.1.2	Seleksi Label dan Kelas Target	24
3.2.1.3	Penyeimbangan Kelas	24
3.2.1.4	Pembagian Dataset dan <i>Group-Aware Split</i>	25
3.2.2	Augmentasi Data	25
3.2.3	Konfigurasi Eksperimen	26
3.2.3.1	Model yang Dibandingkan	26
3.2.3.2	Hiperparameter Pelatihan	27
3.2.3.3	Kriteria Pemilihan Model Terbaik	27
3.2.4	Metrik Evaluasi	28
3.3	Pipeline Ekspor Model ONNX	28
3.3.1	Proses Ekspor	28
3.3.2	Integrasi Aset ke Flutter	30
3.4	Arsitektur Sistem Aplikasi Android	31
3.4.1	Gambaran Umum Sistem (Diagram Konteks C4)	31
3.4.2	Arsitektur Internal (Diagram Kontainer C4)	31
3.4.3	Alur Klasifikasi AI (Diagram Sekuens UML)	34
3.4.4	Model Domain (Diagram Kelas UML)	35
3.4.5	Kasus Penggunaan (Diagram <i>Use Case</i> UML)	37
3.5	Mekanisme Perutean Instansi	38
3.6	Struktur Modul Aplikasi Flutter	39
DAFTAR PUSTAKA		40

DAFTAR TABEL

3.1	Kelas Dataset yang Digunakan	24
3.2	Detail Arsitektur <i>Backbone</i> yang Dibandingkan	26
3.3	Konfigurasi Hiperparameter Pelatihan	27
3.4	Kluster Operasional Perutean Instansi	38
3.5	Modul Fitur Aplikasi Flutter	39

DAFTAR GAMBAR

3.1	Kerangka Berpikir Penelitian	22
3.2	Alur Pengembangan Model AI	23
3.3	Alur Ekspor dan Validasi Model ONNX	29
3.4	Diagram Konteks C4 — Smart City Reporter App	32
3.5	Diagram Kontainer C4 — Komponen Internal Aplikasi	33
3.6	Diagram Sekuens — Alur Klasifikasi AI dan Penyimpanan Laporan	34
3.7	Diagram Kelas — Domain Model Aplikasi	36
3.8	Diagram <i>Use Case</i> — Fungsionalitas Aplikasi	37

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pelaporan masalah lingkungan oleh masyarakat saat ini masih menghadapi hambatan struktural yang signifikan. Berdasarkan analisis data yang dikumpulkan oleh tim peneliti pada portal pengaduan masyarakat kota besar di Indonesia, rata-rata waktu yang dibutuhkan untuk merespons satu laporan masuk—mulai dari penerimaan hingga penugasan ke instansi yang bertanggung jawab—mencapai hampir dua jam. Keterlambatan tersebut bukan semata disebabkan oleh keterbatasan kapasitas instansi, melainkan oleh sifat manual dari proses triase: petugas harus membaca setiap laporan yang masuk, menginterpretasinya, lalu secara manual menentukan instansi yang paling tepat untuk menanganinya. Dengan volume laporan yang terus meningkat seiring dengan pertumbuhan populasi perkotaan, pendekatan manual ini tidak lagi berkelanjutan ([Siret and Sabadie, 2022](#)).

Masalah lingkungan perkotaan seperti jalanan rusak, kecelakaan, sampah menumpuk, vandalisme, dan pohon tumbang memiliki karakteristik yang khas: masalah-masalah ini dapat dideteksi secara visual. Seorang warga yang menemukan lubang besar di jalan tepat di depannya memiliki bukti visual yang jauh lebih kuat daripada deskripsi teks mana pun. Namun, sistem pelaporan berbasis teks atau formulir tidak mampu memanfaatkan informasi visual tersebut; sistem hanya menerima apa yang dituliskan pelapor, yang kerap singkat, ambigu, atau tidak konsisten. Kondisi ini menyebabkan keterlambatan penanganan atau kesalahan dalam prioritas penanganan masalah, terutama ketika laporan yang masuk berjumlah besar ([Firgia et al.,](#)

2022).

Solusi yang muncul secara alami adalah integrasi *Artificial Intelligence* (AI) berbasis pengenalan citra ke dalam sistem pelaporan. Dengan kemajuan arsitektur *deep learning* modern, sebuah model yang telah dilatih dengan data gambar masalah lingkungan mampu mengklasifikasikan jenis masalah secara otomatis hanya dari foto yang diambil oleh pelapor—tanpa memerlukan interpretasi manual dari petugas. Klasifikasi otomatis ini menjadi fondasi untuk penugasan laporan ke instansi yang tepat secara instan, memangkas waktu respons secara dramatis. Hasil penelitian [Shen and Hu \(2025\)](#) menunjukkan bahwa penerapan model pengenalan citra berbasis kecerdasan buatan dalam tata kelola kota mampu meningkatkan akurasi identifikasi kejadian secara signifikan dibandingkan dengan proses manual konvensional, sekaligus mereduksi beban kerja petugas dalam proses kategorisasi laporan.

Kendati demikian, pendekatan berbasis server konvensional—di mana gambar dikirim ke server pusat untuk diproses oleh model AI—menghadirkan tantangan tersendiri: latensi jaringan yang tidak dapat diprediksi, biaya infrastruktur server GPU yang tinggi, serta risiko privasi karena gambar lokasi kejadian harus dikirim ke pihak ketiga. Perkembangan terbaru dalam bidang *on-device machine learning* menawarkan alternatif yang lebih efisien: model AI yang berjalan langsung di perangkat *smartphone* pengguna tanpa memerlukan koneksi internet untuk proses inferensi ([Sadiq and Omlin, 2025](#)).

Kemajuan ini dimungkinkan oleh dua faktor yang berkembang secara bersamaan. Pertama, munculnya arsitektur jaringan saraf visual yang ringan namun sangat akurat—seperti keluarga *Hybrid State Space Models* (MambaOut), *Hierarchical Vision Transformers* (Hiera), *modern ConvNets* (ConvNeXt V2), *Efficient Vision Transformers* (EfficientViT), dan *Foundation Vision Models* (EVA-02)—yang dirancang untuk efisiensi komputasi tanpa mengorbankan akurasi ([Yu and Wang, 2024](#); [Ryali et al., 2023](#); [Liu et al., 2023b](#)). Kedua, standar pertukaran model *Open Neural Network Exchange* (ONNX) yang memungkinkan model yang dilatih di Python dengan PyTorch untuk dieksekusi di platform Android melalui *ONNX Runtime* tanpa modifikasi arsitektur ([Microsoft, 2024](#)).

Dengan mengombinasikan kemajuan ini, dimungkinkan untuk membangun sebuah aplikasi Android yang mengklasifikasikan masalah lingkungan langsung di perangkat pengguna, secara *real-time*, tanpa mengirimkan gambar ke server mana pun. Hasil klasifikasi kemudian secara otomatis merekomendasikan instansi pemerintah daerah yang tepat—seperti Dinas Bina Marga, Dinas Lingkungan Hidup, Satpol PP, Dinas Perhubungan, atau BPBD—untuk ditindaklanjuti. Pendekatan ini selaras dengan visi *smart city* yang tidak hanya memanfaatkan teknologi, tetapi juga membangun ekosistem partisipasi publik yang responsif, efisien, dan dapat diakses secara luas oleh seluruh lapisan masyarakat.

Penelitian ini oleh karenanya mengusulkan pengembangan prototipe sistem pelaporan masalah lingkungan berbasis kecerdasan buatan *on-device* dengan judul: **“PENGEMBANGAN SISTEM PELAPORAN MASALAH LINGKUNGAN DENGAN INTEGRASI DATA GAMBAR DAN TEKS UNTUK MENDUKUNG *SMART CITY*”**.

1.2 Rumusan Masalah

Berdasarkan uraian latar belakang di atas, penelitian ini merumuskan empat pertanyaan penelitian utama:

1. Bagaimana merancang dan membangun sistem pelaporan masalah lingkungan berbasis *mobile* yang mengintegrasikan kecerdasan buatan *on-device* untuk mengklasifikasikan jenis masalah secara otomatis dari gambar?
2. *Backbone* arsitektur jaringan saraf visual manakah—dari kandidat ConvNeXt V2, EfficientViT, MambaOut, Hiera, dan EVA-02—yang menghasilkan performa klasifikasi terbaik pada dataset delapan kelas masalah lingkungan perkotaan?
3. Bagaimana mengeksport model yang telah dilatih ke format ONNX dan mengintegrasikannya ke dalam aplikasi Android Flutter tanpa kehilangan akurasi inferensi yang signifikan?

4. Bagaimana merancang mekanisme perutean laporan otomatis yang dapat merekomendasikan instansi pemerintah daerah yang tepat berdasarkan hasil klasifikasi AI?

1.3 Ruang Lingkup

Ruang lingkup penelitian ini dibatasi sebagai berikut:

1. Jenis Masalah Lingkungan

- (a) Penelitian berfokus pada delapan kategori masalah yang dapat diidentifikasi secara visual, yaitu: *Accident Issues* (kecelakaan lalu lintas), *Littering Garbage on Public Places Issues* (sampah di tempat umum), *Vandalism Issues* (vandalisme), *Pothole Issues* (lubang jalan), *Damaged Road Issues* (kerusakan permukaan jalan), *Broken Road Sign Issues* (rambu lalu lintas rusak), dan *Fallen Tree Issues* (pohon tumbang).
- (b) Kategori *Illegal Parking Issues* dan *Mixed Issues* dikecualikan karena memiliki ambiguitas visual yang tinggi dan berpotensi menurunkan kualitas pelatihan model secara keseluruhan.
- (c) Tidak mencakup masalah lingkungan yang tidak dapat diidentifikasi secara visual, seperti polusi udara dan pencemaran air secara kimiawi.

2. Data

- (a) Data gambar masalah lingkungan diperoleh dari dataset publik yang tersedia di platform HuggingFace Hub (`Programmer-RD-AI/road-issues-detection`).
- (b) Teks deskripsi laporan dikumpulkan sebagai konteks laporan, namun tidak dimasukkan sebagai input ke dalam model AI.

3. Platform dan Teknologi

- (a) Sistem diimplementasikan sebagai aplikasi Android menggunakan *framework* Flutter.

- (b) Inferensi AI berjalan sepenuhnya *on-device* menggunakan *ONNX Runtime for Android*, tanpa ketergantungan pada server eksternal untuk proses klasifikasi.
- (c) Penyimpanan data bersifat lokal menggunakan basis data SQLite melalui *library sqflite*—tidak ada *backend* server.

4. Fitur Utama Aplikasi

- (a) Pengguna dapat mengambil foto atau memilih gambar dari galeri sebagai bukti visual masalah lingkungan.
- (b) Sistem mengklasifikasikan gambar secara otomatis ke salah satu dari delapan kategori, beserta tingkat kepercayaan (*confidence*) dan peringkat prediksi teratas (*top predictions*).
- (c) Berdasarkan hasil klasifikasi, sistem merekomendasikan instansi pemerintah daerah yang paling relevan untuk menindaklanjuti laporan.
- (d) Pengguna dapat mengoreksi hasil klasifikasi secara manual sebelum menyimpan laporan.
- (e) Riwayat laporan beserta koordinat GPS lokasi kejadian disimpan secara lokal di perangkat pengguna.

5. Pengguna

- (a) Masyarakat umum sebagai pelapor masalah lingkungan perkotaan.

6. Limitasi

- (a) Sistem yang dibangun merupakan prototipe dan belum terintegrasi dengan sistem informasi pemerintah daerah yang sesungguhnya.
- (b) Pengujian terbatas pada platform Android; iOS tidak termasuk dalam lingkup penelitian ini.

- (c) Performa model AI bergantung pada kualitas foto yang diambil; gambar yang sangat buram, gelap, atau diambil dari sudut yang tidak representatif dapat menurunkan akurasi klasifikasi.

1.4 Tujuan dan Manfaat Penelitian

1.4.1 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah diuraikan, penelitian ini memiliki empat tujuan utama:

1. Mengembangkan prototipe aplikasi Android untuk pelaporan masalah lingkungan yang mengintegrasikan klasifikasi gambar berbasis AI secara *on-device*.
2. Melakukan studi komparatif terhadap lima arsitektur *backbone* jaringan saraf visual modern—ConvNeXt V2, EfficientViT, MambaOut, Hiera, dan EVA-02—untuk mengidentifikasi model dengan performa terbaik pada domain klasifikasi masalah lingkungan perkotaan, diukur dengan *macro F1-score*.
3. Membangun *pipeline* ekspor model dari PyTorch ke format ONNX dan memvalidasi kesesuaian numerik hasil inferensi antara model PyTorch asli dan model ONNX yang berjalan di Android.
4. Merancang mekanisme perutean laporan otomatis yang memetakan hasil klasifikasi AI ke rekomendasi instansi pemerintah daerah berdasarkan empat kluster operasional.

1.4.2 Manfaat Penelitian

1. **Manfaat bagi Masyarakat:** Aplikasi ini mempermudah proses pelaporan masalah lingkungan melalui panduan klasifikasi otomatis berbasis AI, sehingga laporan yang dihasilkan lebih terstruktur dan informatif bagi instansi terkait.

2. **Manfaat bagi Pemerintah Daerah:** Mekanisme perutean otomatis memiliki potensi untuk memangkas waktu triase manual secara signifikan—dari rata-rata hampir dua jam menjadi hampir instan—memungkinkan laporan didistribusikan ke instansi yang tepat tanpa intervensi manusia.
3. **Manfaat bagi Ilmu Pengetahuan:** Penelitian ini memberikan kontribusi berupa hasil *benchmark* komparatif antara lima arsitektur *backbone* visual modern pada domain spesifik klasifikasi masalah lingkungan perkotaan, serta demonstrasi *pipeline* lengkap dari pelatihan model berbasis PyTorch hingga inferensi *on-device* di Android menggunakan ONNX Runtime.
4. **Manfaat bagi Ekosistem *Smart City*:** Sistem ini membuktikan bahwa kecerdasan buatan yang berjalan di perangkat pengguna—tanpa infrastruktur *cloud*—merupakan jalur yang layak untuk membangun layanan publik cerdas yang inklusif, hemat biaya, dan tidak bergantung pada konektivitas internet yang stabil.

1.5 Metode Penelitian

Metodologi penelitian ini disusun dalam empat tahap utama yang saling berkesinambungan.

1.5.1 Pengumpulan dan Persiapan Data

Dataset gambar masalah lingkungan perkotaan diperoleh dari HuggingFace Hub melalui *snapshot download* ke direktori lokal. Data kemudian diproses melalui tahapan pembersihan label, penyeimbangan kelas menggunakan teknik *balanced sampling* dengan target 800 sampel per kelas, dan pembagian data secara stratifikasi dengan mempertimbangkan kelompok asal gambar (*group-aware stratified split*) untuk mencegah kebocoran data antar partisi *train*, *validation*, dan *test*.

1.5.2 Benchmarking Model AI

Pada tahap ini dilakukan studi komparatif terhadap lima arsitektur *backbone* modern yang diakses melalui *library timm* (*PyTorch Image Models*): ConvNeXt V2 Tiny, EfficientViT B2, MambaOut Tiny, Hiera Tiny, dan EVA-02 Tiny. Setiap model dilatih dengan konfigurasi yang identik—optimizer AdamW, *cosine annealing learning rate*, *early stopping*, dan *automatic mixed precision* FP16—untuk memastikan perbandingan yang adil dan terukur. Seleksi model terbaik didasarkan pada metrik *macro F1-score* pada set validasi.

1.5.3 Ekspor Model ke ONNX dan Validasi

Model terbaik hasil *benchmark* diekspor ke format ONNX menggunakan `torch.onnx.export` dengan *opset* versi 17 dan optimasi *constant folding*. Validasi numerik dilakukan dengan membandingkan *logit* keluaran model PyTorch dan model ONNX pada gambar sampel yang sama. Model beserta metadata normalisasi (*mean*, *std*, nama kelas, pemetaan label ke kategori) dikemas menjadi aset dalam aplikasi Flutter.

1.5.4 Pengembangan Aplikasi Android dengan Flutter

Aplikasi Android dibangun menggunakan *framework* Flutter dengan arsitektur berbasis fitur (*feature-based architecture*), manajemen *state* Riverpod, navigasi deklaratif GoRouter, dan penyimpanan lokal sqflite. Model ONNX diintegrasikan melalui *bridge on-device AI classification service* yang memanfaatkan *ONNX Runtime for Android*. Mekanisme perutean instansi diimplementasikan dalam modul *classification routing* yang memetakan hasil klasifikasi AI ke empat kluster operasional instansi pemerintah daerah.

BAB 2

TINJAUAN PUSTAKA

2.1 Landasan Teori

2.1.1 *Smart City dan Smart Environment*

Smart City secara umum didefinisikan sebagai kerangka kerja perkotaan yang memanfaatkan teknologi informasi dan komunikasi untuk meningkatkan kualitas hidup warga, mengoptimalkan operasional kota, serta memastikan keberlanjutan ekonomi dan lingkungan. Esensi dari *smart city* terletak pada integrasi enam pilar utama, di mana *Smart Governance* dan *Smart Environment* menjadi fondasi penting untuk menciptakan interaksi yang harmonis antara pemerintah dan masyarakat melalui sistem layanan publik yang responsif (Siret and Sabadie, 2022).

Salah satu tantangan terbesar dalam mewujudkan *Smart Environment* adalah pengelolaan laporan masalah lingkungan perkotaan yang datang dari berbagai sumber secara bersamaan. Menurut Sadiq and Omlin (2025), kota-kota pintar bergantung pada infrastruktur penginderaan yang beragam—mulai dari perangkat IoT, kamera pengawas, hingga *smartphone* warga—yang menghasilkan data dalam jumlah masif. Kemampuan untuk memproses dan menginterpretasi data visual dari lingkungan perkotaan secara otomatis merupakan komponen kritis dalam membangun sistem tata kelola kota yang adaptif. Penelitian Shen and Hu (2025) secara khusus mendemonstrasikan bahwa integrasi model pengenalan citra berbasis kecerdasan buatan ke dalam sistem tata kelola kota mampu meningkatkan akurasi identifikasi kejadian secara signifikan dan mengurangi beban kerja manual petugas dalam proses kategorisasi laporan masuk.

2.1.2 Sistem Pelaporan dan Partisipasi Publik

Masalah lingkungan perkotaan yang kompleks—seperti penumpukan sampah, kerusakan fasilitas umum, dan infrastruktur yang buruk—sering kali tidak tertangani dengan cepat akibat keterbatasan metode pemantauan konvensional yang bersifat reaktif dan bergantung pada laporan manual. Akibatnya, dibutuhkan sistem pelaporan masyarakat berbasis teknologi yang memungkinkan warga berpartisipasi aktif dalam penyebaran informasi secara *real-time*.

Hasil penelitian [Firgia et al. \(2022\)](#) menunjukkan bahwa digitalisasi sistem pengaduan melalui platform *mobile* dan web dapat mempercepat pengiriman data dari masyarakat ke pemerintah sekaligus mengurangi kesalahan data yang disebabkan oleh proses manual. Partisipasi publik merupakan komponen esensial dalam pembangunan berkelanjutan; sistem pelaporan yang dirancang dengan baik harus mampu mendorong keterlibatan warga untuk meningkatkan respons pemerintah terhadap ancaman lingkungan. Integrasi kecerdasan buatan ke dalam sistem pelaporan—khususnya melalui klasifikasi citra otomatis—membawa dimensi baru yang memungkinkan laporan diproses, dikategorikan, dan didistribusikan ke instansi yang tepat tanpa intervensi manual dari operator.

2.2 Arsitektur *Transformer* dan *Vision Transformer*

Arsitektur *Transformer*, yang pertama kali diperkenalkan oleh [Vaswani et al. \(2023\)](#), merevolusi cara pemrosesan data sekuensial dengan menggunakan mekanisme *self-attention*. Berbeda dari arsitektur *Convolutional Neural Networks* (CNN) yang terbatas pada pemrosesan fitur lokal dalam jendela reseptif statis, maupun *Recurrent Neural Networks* (RNN) yang menghadapi kendala dalam memodelkan ketergantungan jarak jauh (*long-range dependencies*), *Transformer* mampu menangkap konteks global secara paralel di seluruh elemen input.

Vision Transformer (ViT) ([Dosovitskiy et al., 2021](#)) mengadaptasi paradigma ini ke domain *computer vision* dengan mendekomposisi gambar input menjadi sekumpulan potongan kecil (*patches*) berukuran tetap yang bersifat *non-overlapping*. Se-

tiap *patch* diproyeksikan secara linear menjadi *vector embedding* satu dimensi, berfungsi sebagai urutan token yang analog dengan representasi kata dalam pemrosesan bahasa alami. Mekanisme *self-attention* global kemudian memungkinkan model menangkap hubungan spasial di seluruh bagian gambar secara simultan. ViT menjadi fondasi penting bagi berbagai arsitektur modern yang dikembangkan sesudahnya, termasuk beberapa model yang digunakan dalam penelitian ini.

2.3 Arsitektur *Backbone* Visual Modern

Penelitian ini melakukan studi komparatif terhadap lima arsitektur *backbone* visual modern yang dipilih untuk merepresentasikan berbagai paradigma desain jaringan saraf yang sedang berkembang. Semua model diakses melalui *library timm* (*PyTorch Image Models*) dengan bobot pra-latih (*pre-trained weights*) dari ImageNet.

2.3.1 ConvNeXt V2

ConvNeXt V2 (Liu et al., 2023b) merupakan generasi kedua dari keluarga arsitektur ConvNeXt yang memodernisasi jaringan konvolusi murni (*pure ConvNet*) dengan mengadopsi prinsip desain dari arsitektur *Transformer*. Berbeda dari pendahulunya ConvNeXt V1 yang menggunakan *supervised pre-training*, ConvNeXt V2 dilatih menggunakan kerangka *Fully Convolutional Masked Autoencoder* (FCMAE)—sebuah pendekatan *self-supervised pre-training* yang menggabungkan *masked image modeling* dengan *feature response normalization* baru yang disebut *Global Response Normalization* (GRN).

GRN berperan sebagai mekanisme yang meningkatkan kompetisi antar-fitur dan mengurangi redundansi representasi dalam setiap lapisan, yang merupakan permasalahan yang sering muncul ketika *masked autoencoder* diterapkan pada arsitektur konvolusi. Arsitektur ini tetap sepenuhnya konvolusional tanpa menggunakan mekanisme *attention*, sehingga unggul dari segi efisiensi komputasi. Pada skala *tiny*, ConvNeXt V2 Tiny menunjukkan akurasi yang kompetitif terhadap model

berbasis *Transformer* sekelasnya dengan *throughput* yang lebih tinggi.

2.3.2 EfficientViT

EfficientViT (Liu et al., 2023a) adalah arsitektur *Vision Transformer* yang dirancang dengan tujuan utama efisiensi memori dan kecepatan inferensi pada perangkat keras. Inovasi kunci dari EfficientViT terletak pada modul *Efficient Multi-Scale Attention* (EMA) yang menggantikan mekanisme *self-attention* standar dengan pendekatan *linear attention* berbasis *cascaded group attention*.

Linear attention memungkinkan kompleksitas komputasi yang tumbuh secara linear terhadap panjang sekuens (berbeda dengan *softmax attention* standar yang tumbuh secara kuadratik), sehingga signifikan mengurangi konsumsi memori pada resolusi gambar tinggi. Modul *multi-scale* memungkinkan model untuk secara bersamaan memproses fitur dari berbagai skala spasial, meningkatkan kemampuan representasi fitur hierarkis tanpa biaya komputasi yang besar. Karakteristik ini membuat EfficientViT sangat relevan untuk skenario *deployment* pada perangkat dengan sumber daya terbatas.

2.3.3 MambaOut

MambaOut (Yu and Wang, 2024) adalah arsitektur yang lahir dari eksplorasi teoritis tentang peran *State Space Models* (SSM) dalam pengenalan citra. Arsitektur ini dikembangkan dengan mempertanyakan apakah komponen SSM (pemindaian sekuensial / *sequential scan*) benar-benar diperlukan untuk tugas *image classification*—berbeda dari tugas pemodelan bahasa atau video di mana urutan sekuensial lebih relevan.

MambaOut membuktikan bahwa dengan *menghilangkan* komponen SSM dari blok Mamba dan hanya mempertahankan *gated spatial mixing* serta *channel mixing*, model justru mencapai akurasi yang lebih tinggi pada *benchmark* klasifikasi gambar standar. Hasilnya adalah arsitektur yang secara struktural menyerupai *ConvNet* modern dengan *gated convolution*, namun berakar dari kerangka *State Space*.

MambaOut Tiny—varian yang digunakan dalam penelitian ini—tersedia stabil di *library timm* dan merupakan model dengan performa terbaik dalam eksperimen *benchmark* yang dilakukan, mencapai *test macro F1-score* sebesar **0,9917**.

2.3.4 Hiera

Hiera (*Hierarchical Vision Transformer*) (Ryali et al., 2023) memperkenalkan pendekatan yang berlawanan dengan tren umum yang terus menambahkan kompleksitas ke arsitektur *Vision Transformer*: Hiera justru menyederhanakan ViT dengan *menghilangkan* komponen-komponen yang tidak diperlukan.

Inovasi utama Hiera adalah penghapusan *positional embedding* eksplisit dan penggunaan *pooling attention* hierarkis yang terinspirasi dari *Masked Autoencoders* (MAE). Pooling yang berjalan secara progresif di seluruh lapisan menghasilkan representasi hierarkis seperti piramid fitur pada CNN, namun tetap dalam kerangka *Transformer* yang murni. Hasilnya adalah model yang lebih sederhana, lebih cepat pada waktu pelatihan dan inferensi, serta lebih mudah disesuaikan untuk berbagai skala tanpa kehilangan akurasi yang signifikan.

2.3.5 EVA-02

EVA-02 (Fang et al., 2023) adalah *vision foundation model* generasi kedua dari seri EVA yang menggabungkan dua paradigma pra-latihan yang kuat: *Masked Image Modeling* (MIM) dan penyelarasan dengan representasi CLIP (*Contrastive Language-Image Pre-training*). Inovasi utama EVA-02 terletak pada penggunaan fitur CLIP yang telah dilatih dari data multimodal sebagai target sinyal supervisory dalam proses MIM, menggantikan piksel atau representasi statistik gambar yang biasa digunakan.

Pendekatan ini memaksa model untuk mempelajari representasi semantik yang lebih kaya dan lebih *transferable* dibandingkan model yang dilatih dengan MIM standar. Bahkan pada varian *Tiny* dengan jumlah parameter yang sangat terbatas, EVA-02 mampu menghasilkan fitur visual dengan kualitas yang mendekati *founda-*

tion model berukuran jauh lebih besar, berkat kekayaan pengetahuan semantik yang ditransfer dari sumber CLIP multimodal.

2.4 ONNX dan Inferensi *On-Device*

2.4.1 *Open Neural Network Exchange* (ONNX)

Open Neural Network Exchange (ONNX) adalah standar format terbuka yang dirancang sebagai jembatan interoperabilitas antar-*framework machine learning* (Microsoft, 2024). ONNX mendefinisikan sebuah grafik komputasi (*computation graph*) yang bersifat *framework-agnostic*, memungkinkan model yang dikembangkan dalam PyTorch, TensorFlow, atau *framework* lainnya untuk diekspor ke representasi tunggal yang dapat dieksekusi oleh berbagai *runtime* di berbagai platform.

Representasi ONNX terdiri dari serangkaian *operator* standar yang didefinisikan dalam spesifikasi *opset* bernomor. *Opset* versi 17—yang digunakan dalam penelitian ini—mendukung set operator yang komprehensif dan stabil, termasuk semua operator yang diperlukan oleh arsitektur MambaOut Tiny. Saat ekspor, PyTorch melakukan *tracing* grafik komputasi model dan menerjemahkannya ke operator ONNX yang setara, dengan opsi optimasi seperti *constant folding* yang mengeliminasi subgraf konstan untuk mengurangi ukuran model dan latensi inferensi.

2.4.2 *ONNX Runtime for Android*

ONNX Runtime adalah *inference engine* lintas platform yang dioptimalkan untuk mengeksekusi model ONNX secara efisien (Microsoft, 2024). Untuk platform Android, *ONNX Runtime* tersedia sebagai paket AAR (*Android Archive*) yang dapat diintegrasikan langsung ke dalam proyek Android melalui *dependency manager* Gradle. *Runtime* ini mendukung akselerasi perangkat keras melalui berbagai *execution provider*—termasuk NNAPI (*Android Neural Networks API*) untuk akselerasi *on-chip*—yang memungkinkan model berukuran kecil berjalan dengan latensi yang dapat diterima bahkan pada perangkat kelas menengah.

Pola *companion metadata* juga menjadi praktik penting dalam *deployment* ON-

NX: sebuah file JSON yang diekspor bersamaan dengan model ONNX menyimpan informasi normalisasi (*mean* dan *std* setiap kanal warna), nama kelas keluaran, serta pemetaan dari label dataset mentah ke kategori semantik aplikasi. Pemisahan meta-data dari bobot model memungkinkan pembaruan nama kelas atau logika pemetaan tanpa perlu mengekspor ulang model ONNX.

2.5 Flutter dan Arsitektur Aplikasi *Mobile*

2.5.1 Flutter

Flutter adalah *framework* pengembangan aplikasi lintas platform berbasis bahasa Dart yang dikembangkan oleh Google ([Google LLC, 2024](#)). Berbeda dari pendekatan *hybrid* yang membungkus komponen antarmuka platform (seperti React Native), Flutter menggunakan *rendering engine* sendiri (Skia/Impeller) untuk menggambar setiap piksel, menghasilkan tampilan yang konsisten di semua platform tanpa bergantung pada komponen antarmuka asli (*native widgets*).

Dalam konteks penelitian ini, Flutter dipilih karena kemampuannya mengintegrasikan kode Dart dengan kode platform asli (Java/Kotlin untuk Android) melalui mekanisme *Platform Channel*. Integrasi *ONNX Runtime*—yang menyediakan API Java/Kotlin—ke dalam aplikasi Flutter dilakukan melalui *bridge* ini, memungkinkan inferensi model berjalan dalam konteks Android asli dengan performa optimal sambil tetap menggunakan antarmuka pengguna yang dibangun sepenuhnya di Dart.

2.5.2 Riverpod

Riverpod adalah *library* manajemen *state* untuk Flutter yang menawarkan keamanan tipe kompilasi (*compile-time type safety*) dan reaktivitas yang fleksibel ([Rousset, 2024](#)). Berbeda dari pendahulunya Provider, Riverpod tidak bergantung pada hierarki *widget* untuk mendistribusikan *state*, melainkan menggunakan konsep *provider* yang berdiri sendiri dan dapat diakses dari mana saja dalam kode aplikasi.

Dalam arsitektur aplikasi penelitian ini, Riverpod digunakan sebagai lapisan manajemen *state* utama yang menjembatani antara lapisan *service* (inferensi ON-

NX, akses basis data, lokasi GPS) dan lapisan antarmuka pengguna. Pola *provider* Riverpod memungkinkan setiap komponen aplikasi bereaksi secara otomatis terhadap perubahan *state*—misalnya, hasil klasifikasi yang baru saja diterima dari model ONNX—tanpa memerlukan pengelolaan *lifecycle* yang eksplisit.

2.5.3 GoRouter

GoRouter adalah *library* navigasi deklaratif untuk Flutter yang menyediakan sistem navigasi berbasis *URL path* yang serupa dengan navigasi web. Pendekatan deklaratif ini memungkinkan alur navigasi aplikasi—dari layar laporan baru, ke layar hasil AI, ke layar konfirmasi instansi—didefinisikan secara terpusat dalam satu konfigurasi *router*, meningkatkan keterbacaan kode dan memudahkan pengelolaan alur *deep link*.

2.5.4 SQLite dan sqflite

SQLite adalah sistem manajemen basis data relasional yang ringan dan *server-less*, yang disematkan langsung ke dalam proses aplikasi. Penggunaan SQLite memungkinkan aplikasi menyimpan data secara persisten di perangkat tanpa memerlukan koneksi ke server basis data eksternal—pendekatan yang sesuai dengan desain sistem penelitian ini yang sepenuhnya lokal. Pada Flutter, SQLite diakses melalui *library* sqflite yang menyediakan API asinkron Dart untuk operasi CRUD pada tabel `users`, `reports`, dan `report_history`.

2.6 Metrik Evaluasi Klasifikasi Multi-Kelas

2.6.1 Macro F1-Score

Dalam konteks klasifikasi multi-kelas dengan distribusi kelas yang tidak sepenuhnya seimbang, *macro F1-score* merupakan metrik evaluasi yang lebih informatif dibandingkan akurasi sederhana. Sementara akurasi menghitung rasio prediksi benar terhadap total sampel—yang dapat terlihat tinggi meski model gagal pada kelas

minoritas—*macro F1-score* memberikan bobot yang setara kepada setiap kelas tanpa memperhatikan frekuensi relatifnya.

F1-score untuk setiap kelas c didefinisikan sebagai rata-rata harmonik antara *precision* dan *recall*:

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (2.1)$$

di mana:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (2.2)$$

Macro F1-score kemudian dihitung sebagai rata-rata aritmetika dari $F1_c$ seluruh kelas K :

$$\text{Macro F1} = \frac{1}{K} \sum_{c=1}^K F1_c \quad (2.3)$$

Dalam penelitian ini, *macro F1-score* digunakan sebagai kriteria utama pemilihan *checkpoint* model terbaik selama pelatihan (berdasarkan nilai pada set validasi) maupun peringkat akhir dalam *leaderboard* perbandingan model (berdasarkan nilai pada set uji).

2.6.2 Metrik Pendukung

Selain *macro F1-score*, beberapa metrik pendukung juga dihitung untuk memberikan gambaran evaluasi yang lebih komprehensif:

1. **Akurasi (*Accuracy*)**: Proporsi prediksi benar dari keseluruhan sampel. Berguna sebagai indikator umum performa model.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.4)$$

2. ***Balanced Accuracy***: Rata-rata *recall* per kelas. Lebih robust terhadap ketidakseimbangan kelas dibandingkan akurasi standar.

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{c=1}^K \text{Recall}_c \quad (2.5)$$

3. **Macro ROC-AUC:** Rata-rata *Area Under the Receiver Operating Characteristic Curve* untuk setiap kelas dalam skenario *one-vs-rest*. Mengukur kemampuan model membedakan setiap kelas dari semua kelas lainnya secara probabilistik.
4. **Confusion Matrix:** Matriks $K \times K$ yang memvisualisasikan distribusi prediksi benar dan salah untuk setiap pasangan kelas, memungkinkan identifikasi pola kesalahan spesifik antar kelas yang saling memiliki kemiripan visual.

2.7 Teknik Pelatihan Model

2.7.1 Transfer Learning dan Fine-tuning

Transfer learning adalah paradigma pelatihan di mana bobot model yang telah dilatih pada dataset skala besar (umumnya ImageNet dengan 1,2 juta gambar dan 1.000 kelas) digunakan sebagai inisialisasi untuk melatih model pada dataset target yang lebih kecil (Dosovitskiy et al., 2021). Pendekatan ini terbukti secara konsisten menghasilkan konvergensi yang lebih cepat dan akurasi akhir yang lebih tinggi dibandingkan pelatihan dari awal (*training from scratch*), terutama ketika data target terbatas.

Dalam penelitian ini, seluruh model diinisialisasi dengan bobot pra-latih dari ImageNet melalui API `timm` (`pretrained=True`). Proses *fine-tuning* dilakukan dengan melatih ulang seluruh bobot model—bukan hanya lapisan kepala klasifikasi—menggunakan *learning rate* yang kecil (3×10^{-4}) untuk menjaga representasi fitur yang telah dipelajari selama pra-latih sambil mengadaptasinya ke domain masalah lingkungan perkotaan.

2.7.2 AdamW dan Cosine Annealing

Optimizer AdamW (Loshchilov and Hutter, 2019) merupakan varian dari Adam yang memisahkan penerapan *weight decay* dari pembaruan adaptif gradien. Dalam Adam standar, *weight decay* diterapkan melalui regularisasi L2 yang diintegrasikan ke dalam gradien, sehingga berinteraksi dengan mekanisme adaptif dan menghasilkan efek regularisasi yang berbeda dari yang diharapkan. AdamW memperbaiki hal ini dengan menerapkan *weight decay* secara langsung pada bobot, terlepas dari gradien, menghasilkan regularisasi yang lebih bersih dan performa yang lebih baik pada sebagian besar skenario *fine-tuning*.

Cosine Annealing Learning Rate Scheduler menjadwalkan penurunan *learning rate* mengikuti fungsi kosinus dari nilai awal hingga mendekati nol sepanjang T_{max} epoch. Penurunan bertahap ini memungkinkan model menjelajahi ruang parameter secara halus pada tahap akhir pelatihan, meningkatkan konvergensi ke titik minimum yang lebih baik.

2.7.3 Automatic Mixed Precision (AMP)

Automatic Mixed Precision adalah teknik yang secara otomatis memilih apakah suatu operasi dalam grafik komputasi dieksekusi dengan presisi penuh (*float32*) atau presisi setengah (*float16*), berdasarkan sensitivitas numerik operasi tersebut. Operasi yang sensitif terhadap presisi—seperti fungsi aktivasi dan normalisasi—tetap dijalankan dalam *float32*, sementara operasi yang dominan seperti perkalian matriks dijalankan dalam *float16*. Hasilnya adalah percepatan pelatihan yang signifikan pada GPU modern dengan CUDA tanpa kehilangan akurasi model, karena *GradScaler* secara dinamis menyesuaikan skala gradien untuk mencegah *underflow*.

2.7.4 Label Smoothing

Label smoothing adalah teknik regularisasi yang mengubah label *one-hot* keras menjadi distribusi probabilitas halus: label target yang benar mendapatkan probabilitas $1 - \epsilon$, sementara sisa probabilitas ϵ didistribusikan secara merata ke semua

kelas lainnya. Dengan nilai $\varepsilon = 0,05$ yang digunakan dalam penelitian ini, teknik ini mencegah model dari terlalu yakin (*overconfident*) terhadap prediksinya, mengurangi *overfitting*, dan meningkatkan generalisasi pada data uji (Szegedy et al., 2016).

BAB 3

METODE PENELITIAN

3.1 Kerangka Berpikir

Penelitian ini diorganisasikan dalam dua fase pengembangan utama yang berurutan namun saling bergantung: (1) fase eksperimen dan optimasi model AI, dan (2) fase pengembangan aplikasi Android. Gambar 3.1 menyajikan kerangka berpikir penelitian secara menyeluruh.

Fase pertama dimulai dari pengumpulan dan persiapan dataset dari HuggingFace Hub, dilanjutkan dengan studi komparatif lima arsitektur *backbone* modern melalui pelatihan dan evaluasi sistematis, hingga pemilihan model terbaik. Fase kedua mencakup ekspor model terpilih ke format ONNX, integrasi ke dalam aplikasi Android Flutter, dan pengujian fungsional sistem secara menyeluruh.

3.2 Pengembangan Model *Artificial Intelligence*

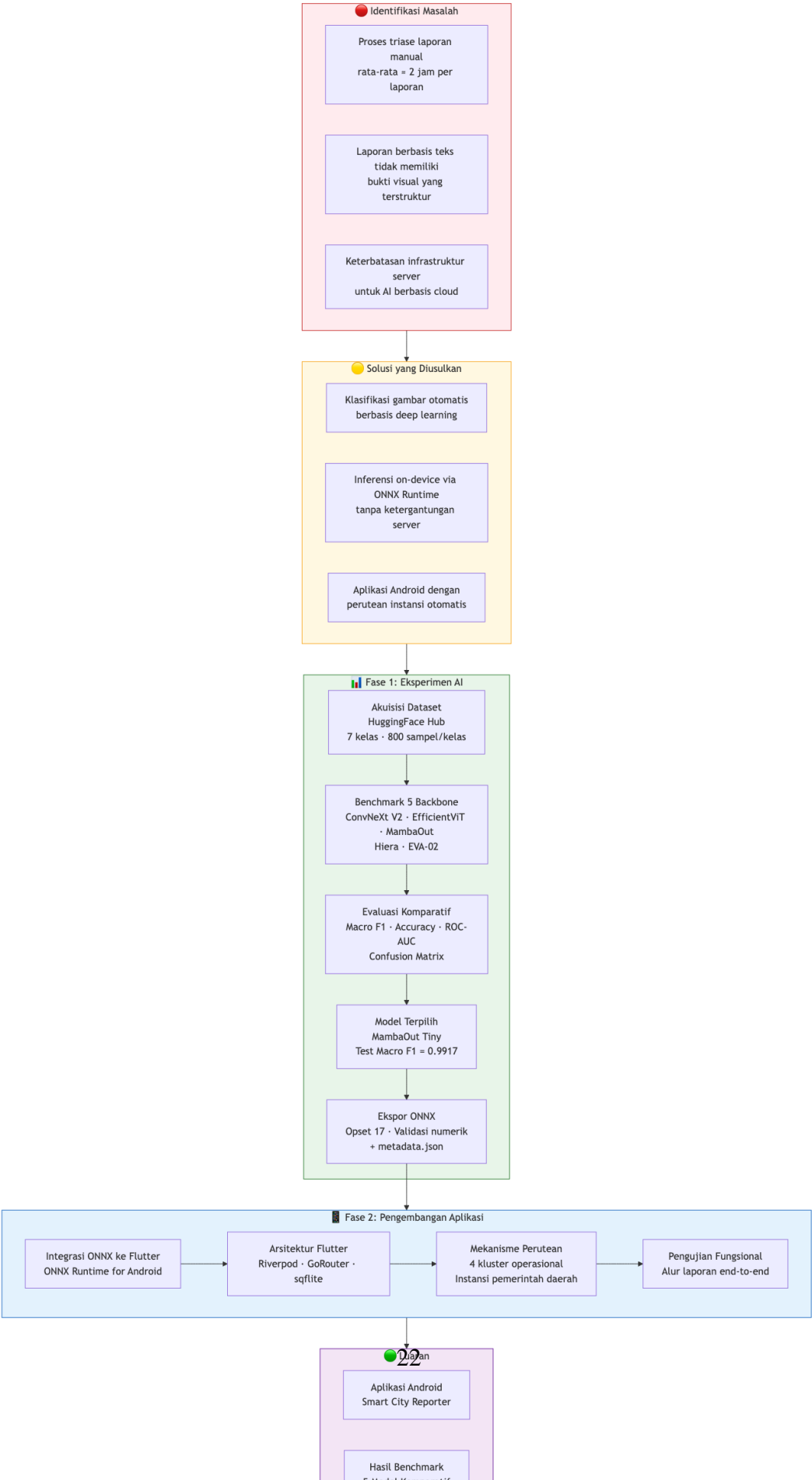
Alur pengembangan kecerdasan buatan dalam penelitian ini dipaparkan secara sistematis pada Gambar 3.2.

3.2.1 Dataset

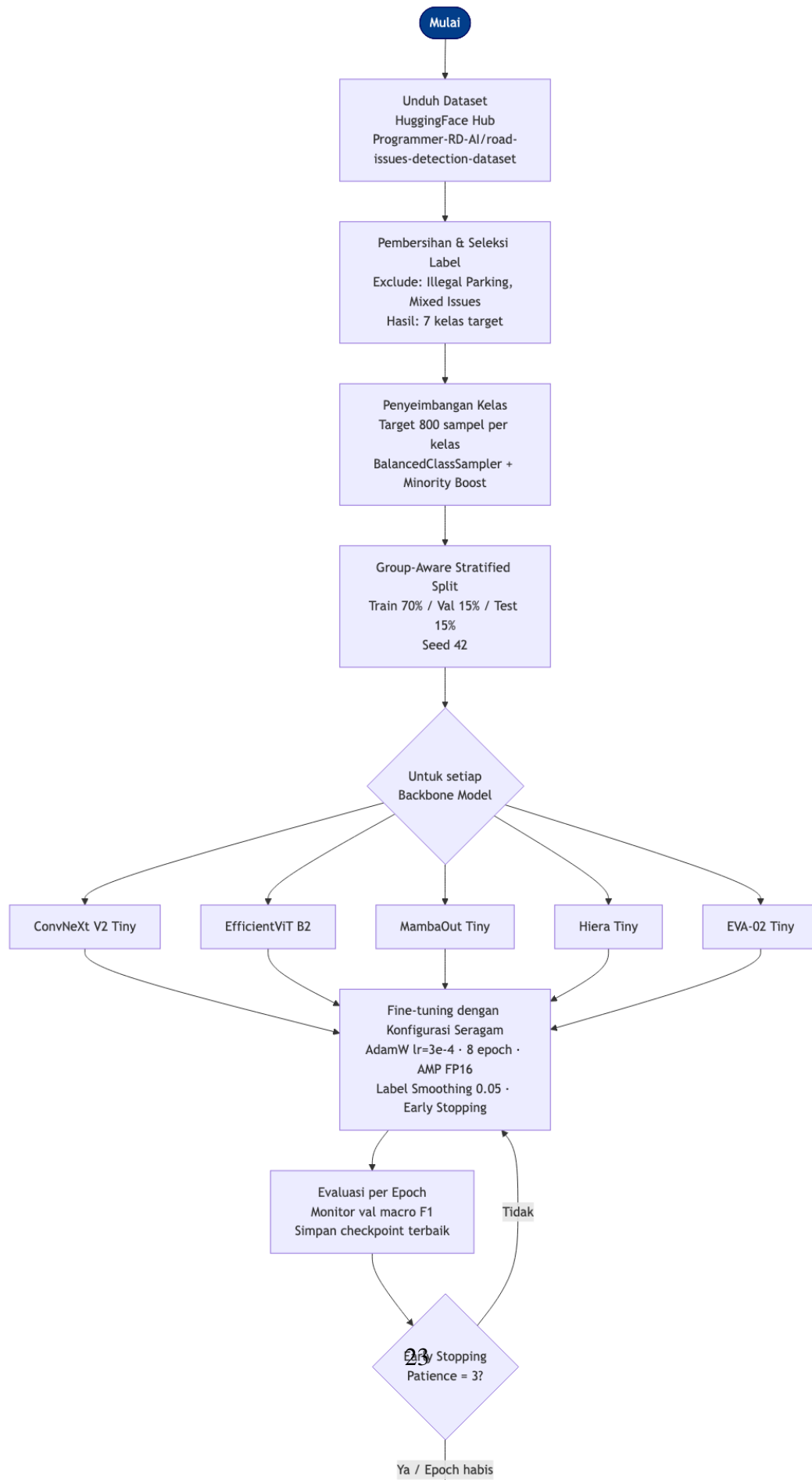
3.2.1.1 Sumber Data

Data yang digunakan dalam penelitian ini diperoleh dari dataset publik yang tersedia di platform HuggingFace Hub dengan identitas repositori `Programmer-RD-AI/road-is`. Dataset ini berisi gambar-gambar kondisi lingkungan perkotaan yang telah dilabeli ke dalam berbagai kategori masalah. Proses pengunduhan dilakukan menggunakan

Kerangka Berpikir Penelitian



Alur Pengembangan Model AI



fungsi `snapshot_download` dari *library* `huggingface_hub`, yang mengunduh seluruh isi repositori dataset—termasuk gambar dan metadata—ke direktori lokal.

3.2.1.2 Seleksi Label dan Kelas Target

Dari keseluruhan label yang tersedia dalam dataset, penelitian ini menggunakan **tujuh kelas** masalah lingkungan perkotaan yang memiliki representasi visual yang cukup jelas dan tidak ambigu. Dua kategori dikecualikan secara eksplisit:

- ***Illegal Parking Issues***: Dikecualikan karena ambiguitas visual yang tinggi—gambar parkir liar seringkali sulit dibedakan dari kondisi normal jalan pada resolusi yang terbatas.
- ***Mixed Issues***: Dikecualikan karena merupakan label campuran yang tidak merepresentasikan satu kategori masalah tunggal, sehingga tidak cocok untuk tugas klasifikasi multi-kelas yang definitif.

Tabel 3.1 merinci kelas-kelas yang digunakan beserta deskripsi singkatnya.

Tabel 3.1. Kelas Dataset yang Digunakan

No.	Label Dataset	Deskripsi
1	<i>Accident Issues</i>	Kecelakaan lalu lintas di jalan
2	<i>Littering Garbage on Public Places Issues</i>	Sampah di tempat umum
3	<i>Vandalism Issues</i>	Vandalisme pada fasilitas publik
4	<i>Pothole Issues</i>	Lubang pada permukaan jalan
5	<i>Damaged Road Issues</i>	Kerusakan permukaan jalan
6	<i>Broken Road Sign Issues</i>	Rambu lalu lintas yang rusak
7	<i>Fallen Tree Issues</i>	Pohon tumbang

3.2.1.3 Penyeimbangan Kelas

Dataset asli memiliki distribusi kelas yang tidak seimbang, dengan beberapa kategori memiliki sampel jauh lebih banyak dari yang lain. Untuk mengatasi hal

ini, penelitian ini menerapkan strategi penyeimbangan dengan **target 800 sampel per kelas** (*target_samples_per_class = 800*). Bagi kelas yang memiliki lebih dari 800 sampel, dilakukan pengambilan sampel acak (*undersampling*). Bagi kelas minoritas yang memiliki kurang dari 800 sampel, teknik *oversampling* implisit diterapkan melalui *BalancedClassSampler* dan augmentasi khusus kelas minoritas selama pelatihan.

3.2.1.4 Pembagian Dataset dan *Group-Aware Split*

Dataset dibagi menjadi tiga partisi dengan rasio: **70% pelatihan (*train*)**, **15% validasi (*validation*)**, dan **15% pengujian (*test*)**, dengan *seed* acak yang ditetapkan pada nilai 42 untuk reproduktibilitas.

Aspek kritis dalam pembagian dataset ini adalah penerapan *group-aware stratified split*. Beberapa gambar dalam dataset merupakan gambar yang diaugmentasi (diperbesar secara sintesis) dari gambar sumber yang sama, khususnya untuk kelas kecelakaan (*Accident Issues*). Jika gambar asli dan turunan augmentasinya dimasukkan ke partisi yang berbeda (*train* dan *test*), model akan mendapatkan gambaran yang terlalu optimis tentang kemampuan generalisasinya—fenomena ini disebut *data leakage*.

Untuk mencegah hal ini, setiap gambar augmentasi ditandai dengan *group key* yang menunjukkan gambar sumber asalnya (ditandai dengan penanda `--source--` pada nama file). Pembagian dataset kemudian dilakukan pada tingkat grup, bukan tingkat gambar individual, sehingga seluruh gambar yang berasal dari satu sumber yang sama selalu berada dalam partisi yang sama.

3.2.2 Augmentasi Data

Augmentasi data diterapkan secara berbeda antara set pelatihan dan set evaluasi:

1. **Transformasi Pelatihan (Training Transform)**: Diterapkan secara acak pada setiap gambar dalam set pelatihan untuk meningkatkan keberagaman data. Mencakup: *random horizontal flip*, *random vertical flip*, *random crop* dengan

padding, *random color jitter* (kecerahan, kontras, saturasi), serta normalisasi menggunakan *mean* dan *std* yang spesifik untuk setiap model backbone (diambil dari `timm data_config`).

2. **Transformasi Kelas Minoritas (Minority Transform):** Kelas yang memiliki rasio jumlah sampel terhadap target di bawah ambang 60% (*minority_threshold_ratio* = 0.6) mendapatkan transformasi augmentasi yang lebih agresif. Ini bertujuan untuk secara efektif melipatgandakan keberagaman visual kelas minoritas tanpa pengulangan data yang identik.
3. **Transformasi Evaluasi (Evaluation Transform):** Hanya berisi *center crop*, *resize*, dan normalisasi—tanpa augmentasi acak—untuk memastikan evaluasi yang deterministik dan reproduibel.

3.2.3 Konfigurasi Eksperimen

3.2.3.1 Model yang Dibandingkan

Lima arsitektur *backbone* visual modern yang digunakan dalam studi komparatif ini, beserta detail teknisnya, disajikan dalam Tabel 3.2.

Tabel 3.2. Detail Arsitektur *Backbone* yang Dibandingkan

Nama Model	Kunci <code>timm</code>	Keluarga	Tahun
ConvNeXt V2 Tiny	<code>convnextv2_tiny</code>	ConvNet	2023
EfficientViT B2	<code>efficientvit_b2</code>	Efficient Transformer	2023
MambaOut Tiny	<code>mambaout_tiny</code>	Hybrid State Space	2024
Hiera Tiny	<code>hiera_tiny_224</code>	Hierarchical ViT	2023
EVA-02 Tiny	<code>eva02_tiny_patch14_224</code>	Foundation ViT	2023

Semua model diinisialisasi dengan bobot pra-latih (*pretrained* = *True*) dari ImageNet melalui *library timm*. Lapisan kepala klasifikasi (*classification head*) setiap model diganti dengan lapisan *linear* baru yang sesuai dengan jumlah kelas target.

3.2.3.2 Hiperparameter Pelatihan

Konfigurasi hiperparameter yang digunakan identik untuk semua model, memastikan perbandingan yang adil. Tabel 3.3 merangkum seluruh hiperparameter yang digunakan.

Tabel 3.3. Konfigurasi Hiperparameter Pelatihan

Hiperparameter	Nilai
Optimizer	AdamW
<i>Learning rate</i>	3×10^{-4}
<i>Weight decay</i>	5×10^{-2}
<i>Label smoothing</i>	0,05
<i>Gradient clip norm</i>	1,0
Epoch maksimum	8
<i>Batch size</i>	24
<i>Early stopping patience</i>	3 epoch
<i>Drop path rate</i>	0,1
<i>LR Scheduler</i>	Cosine Annealing
<i>Automatic Mixed Precision</i>	FP16 (CUDA)
<i>Pretrained weights</i>	ImageNet (via <code>timm</code>)
<i>Balanced Sampler</i>	Diaktifkan
<i>Minority Boost</i>	Diaktifkan (threshold 0,6)

3.2.3.3 Kriteria Pemilihan Model Terbaik

Pemilihan *checkpoint* model terbaik selama pelatihan dan peringkat akhir *leader-board* didasarkan pada ***macro F1-score*** pada set validasi. Pilihan ini didasari oleh kemampuan *macro F1-score* dalam memberikan bobot yang setara kepada semua kelas, sehingga model yang unggul di semua kelas—bukan hanya kelas mayoritas—akan mendapatkan skor yang lebih tinggi.

3.2.4 Metrik Evaluasi

Evaluasi komprehensif dilakukan pada set validasi dan set uji menggunakan metrik berikut (lihat Bab 2 untuk definisi matematis):

- **Macro F1-score** (metrik utama)
- **Akurasi (Accuracy)**
- **Balanced Accuracy**
- **Macro ROC-AUC, Weighted ROC-AUC, Micro ROC-AUC**
- **Confusion Matrix** (untuk analisis per-kelas)
- **Presisi, Recall, F1 per-kelas**

3.3 Pipeline Ekspor Model ONNX

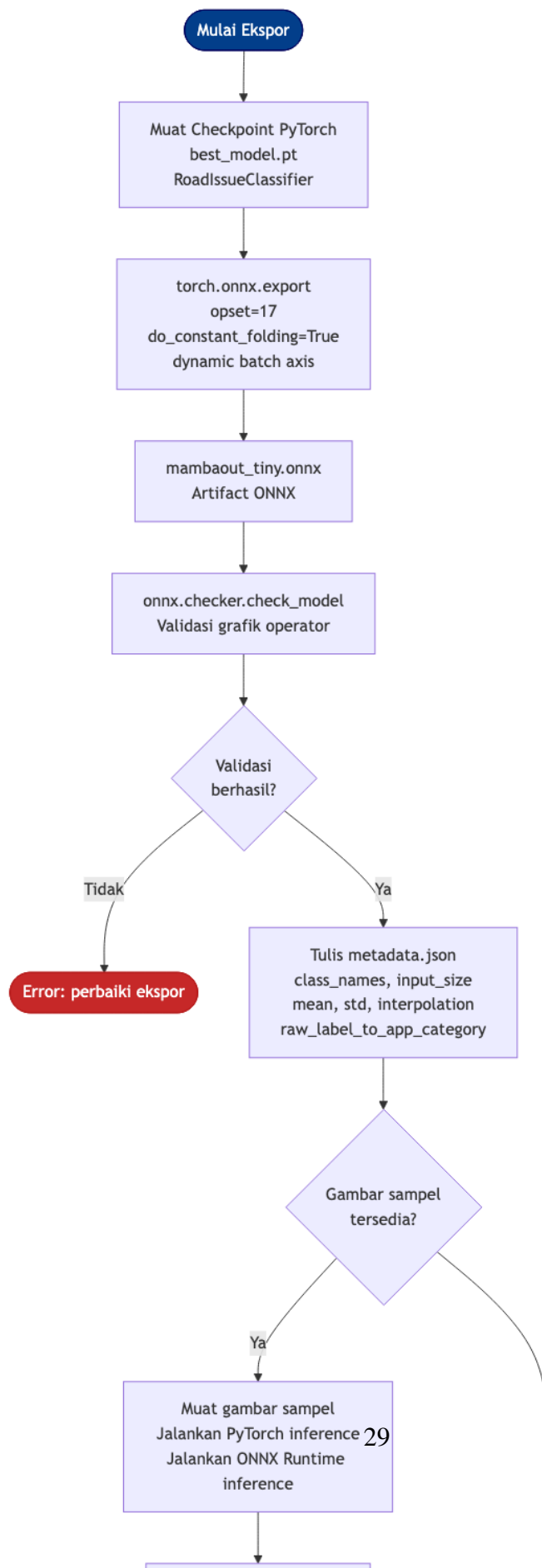
Setelah model terbaik diidentifikasi, dilakukan ekspor ke format ONNX untuk mempersiapkan *deployment* di aplikasi Android. Gambar 3.3 mengilustrasikan alur ekspor ini.

3.3.1 Proses Ekspor

Ekspor dilakukan melalui skrip `export_onnx.py` dengan langkah-langkah berikut:

1. **Pemuatan *Checkpoint*:** *Checkpoint* model PyTorch (`best_model.pt`) dimuat ke dalam kelas `RoadIssueClassifier`, yang mengenkapsulasi model, spesifikasi arsitektur, nama kelas, dan konfigurasi normalisasi data.
2. **Ekspor ONNX:** Fungsi `torch.onnx.export` digunakan dengan konfigurasi:
 - *Opset* versi 17 (untuk kompatibilitas dengan *ONNX Runtime* versi terkini)

Alur Ekspor dan Validasi Model ONNX



- `do_constant_folding = True`: mengeliminasi subgraf konstan pada *compile time*
 - Nama input: `input`; nama output: `logits`
 - *Dynamic batch axis*: memungkinkan inferensi dengan ukuran *batch* yang bervariasi
3. **Validasi Model ONNX**: Model ONNX yang dihasilkan divalidasi menggunakan `onnx.checker.check_model` untuk memastikan konsistensi grafik operator.
 4. **Ekspor Metadata**: File `.metadata.json` ditulis bersamaan dengan file ONNX, berisi:
 - Kunci dan nama model
 - Nama kelas keluaran (`class_names`)
 - Ukuran input (`input_size`), *mean* dan *std* normalisasi
 - Pemetaan dari label dataset mentah ke kategori semantik aplikasi (`raw_label_to_app_c`)
 5. **Validasi Paritas Numerik**: Jika gambar sampel tersedia, logit dari model PyTorch dan model ONNX dibandingkan. Perbedaan absolut maksimum (*max abs diff*) dihitung dan dicatat dalam laporan JSON. Nilai ambang yang dapat diterima adalah di bawah 10^{-5} , yang memastikan bahwa prediksi kelas teratas (*top-1*) antara kedua model selalu identik.

3.3.2 Integrasi Aset ke Flutter

File `mambaout_tiny.onnx` dan `mambaout_tiny.metadata.json` yang dihasilkan dikemas ke dalam direktori aset aplikasi Flutter di `assets/models/`. Pada saat *build* aplikasi, Flutter akan menyertakan file-file ini ke dalam paket APK Android, memungkinkan *ONNX Runtime* membaca model langsung dari aset tanpa memerlukan akses internet atau unduhan tambahan.

3.4 Arsitektur Sistem Aplikasi Android

3.4.1 Gambaran Umum Sistem (Diagram Konteks C4)

Gambar 3.4 mengilustrasikan konteks sistem pada level tertinggi, menunjukkan aktor dan sistem eksternal yang berinteraksi dengan aplikasi.

Sistem memiliki satu aktor utama—**Masyarakat** sebagai pelapor—yang berinteraksi dengan aplikasi melalui antarmuka pengguna. Aplikasi mengakses dua layanan eksternal perangkat: **GPS/Location API** untuk memperoleh koordinat geografis lokasi kejadian, dan **Kamera/Galeri** untuk mengambil foto bukti masalah. Tidak ada interaksi dengan server eksternal dalam alur utama, menjadikan sistem sepenuhnya *offline-capable* untuk fungsi pelaporan dan klasifikasi.

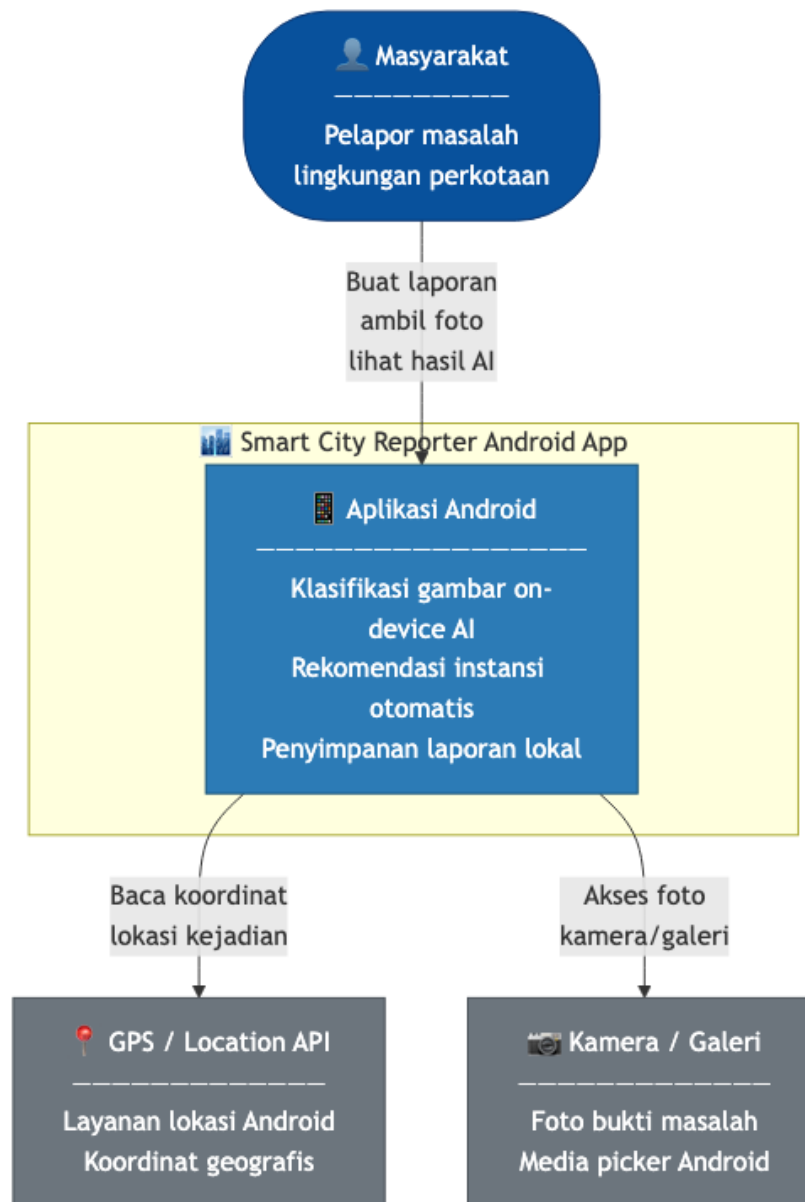
3.4.2 Arsitektur Internal (Diagram Kontainer C4)

Gambar 3.5 merinci komponen-komponen utama di dalam aplikasi beserta hubungannya.

Arsitektur internal aplikasi terdiri dari delapan komponen utama:

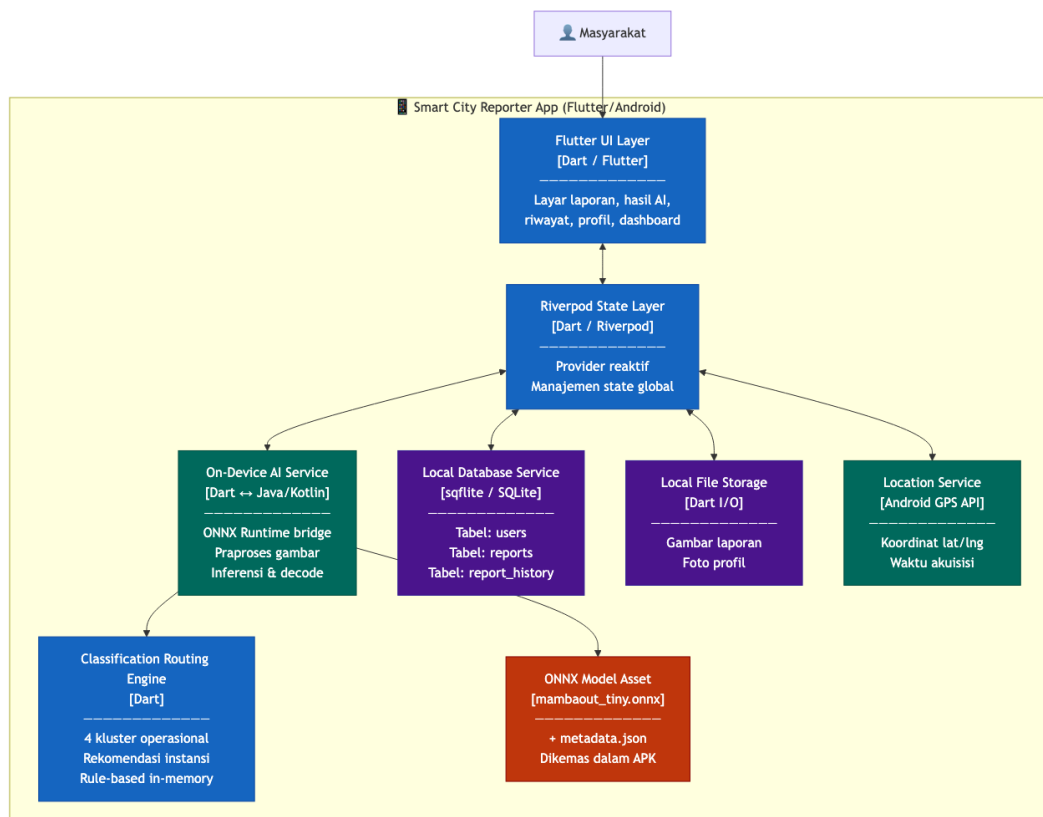
1. **Flutter UI Layer:** Lapisan antarmuka pengguna yang dibangun sepenuhnya dalam Dart menggunakan *framework* Flutter. Terdiri dari layar-layar dalam modul fitur (`features/`).
2. **Riverpod State Layer:** Lapisan manajemen *state* yang menjembatani antara UI dan *service layer*. Setiap *provider* merepresentasikan satu unit *state* atau *service* yang dapat diobservasi oleh widget.
3. **On-Device AI Service** (`on_device_ai_classification_service.dart`): *Bridge* antara Flutter (Dart) dan *ONNX Runtime* (Android). Memuat model ONNX dari aset, melakukan pra-pemrosesan gambar (*resize*, *crop*, *normalisasi*), menjalankan sesi inferensi, dan menerjemahkan *logit* keluaran menjadi objek `AiPrediction`.

Diagram Konteks C4 — Smart City Reporter App



Gambar 3.4. Diagram Konteks C4 — Smart City Reporter App

Diagram Kontainer C4 – Komponen Internal Aplikasi

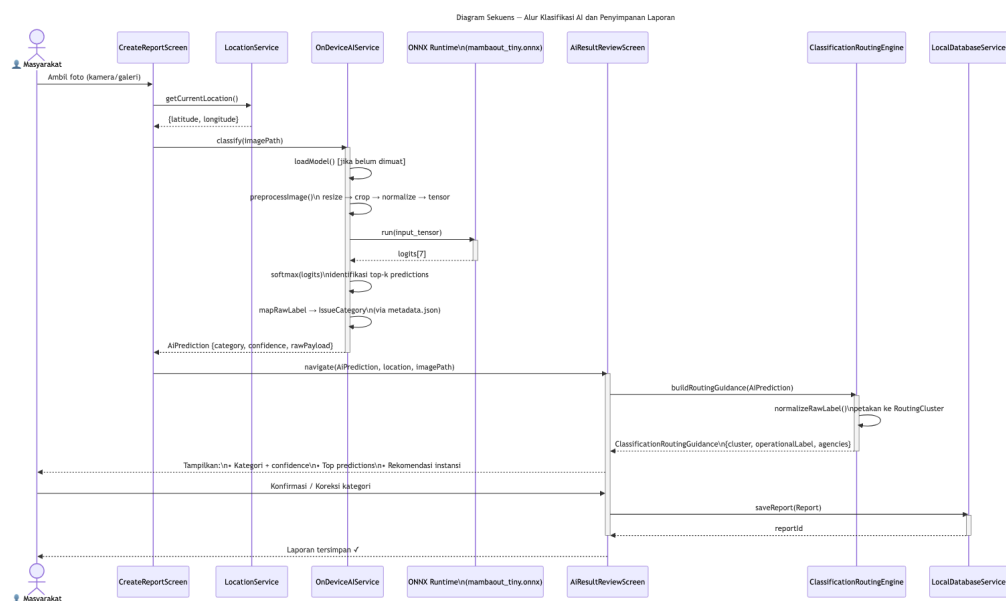


Gambar 3.5. Diagram Kontainer C4 — Komponen Internal Aplikasi

4. **Classification Routing Engine** (`classification_routing.dart`): Modul yang memetakan hasil klasifikasi AI ke panduan perutean instansi. Beroperasi secara murni *in-memory* tanpa akses jaringan.
5. **Local Database Service** (`local_database_service.dart`): Lapisan akses SQLite melalui sqflite. Mengelola tabel `users`, `reports`, dan `report_history`.
6. **Local File Storage Service** (`local_file_storage_service.dart`): Mengelola penyimpanan file gambar laporan dan foto profil di direktori sistem Android.
7. **Location Service** (`location_service.dart`): Mengakses GPS perangkat untuk memperoleh koordinat geografis lokasi laporan.
8. **ONNX Model Asset**: File `mambaout_tiny.onnx` dan `mambaout_tiny.metadata.js` yang dikemas dalam APK.

3.4.3 Alur Klasifikasi AI (Diagram Sekuens UML)

Gambar 3.6 mengilustrasikan alur terinci dari proses klasifikasi AI, mulai dari pengguna mengambil foto hingga laporan tersimpan.



Gambar 3.6. Diagram Sekuens — Alur Klasifikasi AI dan Penyimpanan Laporan

Alur proses klasifikasi berlangsung sebagai berikut:

1. Pengguna mengambil foto melalui kamera atau memilih gambar dari galeri pada `CreateReportScreen`.
2. `CreateReportScreen` memanggil metode `classify(imagePath)` pada `OnDeviceAIService`.
3. `OnDeviceAIService` melakukan pra-pemrosesan gambar: resize ke ukuran input model, normalisasi piksel, dan konversi ke tensor.
4. Tensor input dikirim ke *ONNX Runtime* yang mengeksekusi inferensi pada model `mambaout_tiny.onnx`.
5. *ONNX Runtime* mengembalikan *logit* untuk ketujuh kelas.
6. `OnDeviceAIService` menerapkan fungsi *softmax*, mengidentifikasi prediksi teratas, dan membungkus hasil dalam objek `AiPrediction`.
7. Pengguna dialihkan ke `AiResultReviewScreen` yang menampilkan hasil klasifikasi beserta tingkat kepercayaan dan visual *top predictions*.
8. Secara paralel, `ClassificationRoutingGuidance` dibangun dari `AiPrediction`, menghasilkan daftar rekomendasi instansi dan kluster operasional.
9. Pengguna dapat mengonfirmasi atau mengoreksi kategori secara manual.
10. Setelah konfirmasi, laporan disimpan ke SQLite melalui `LocalDatabaseService`.

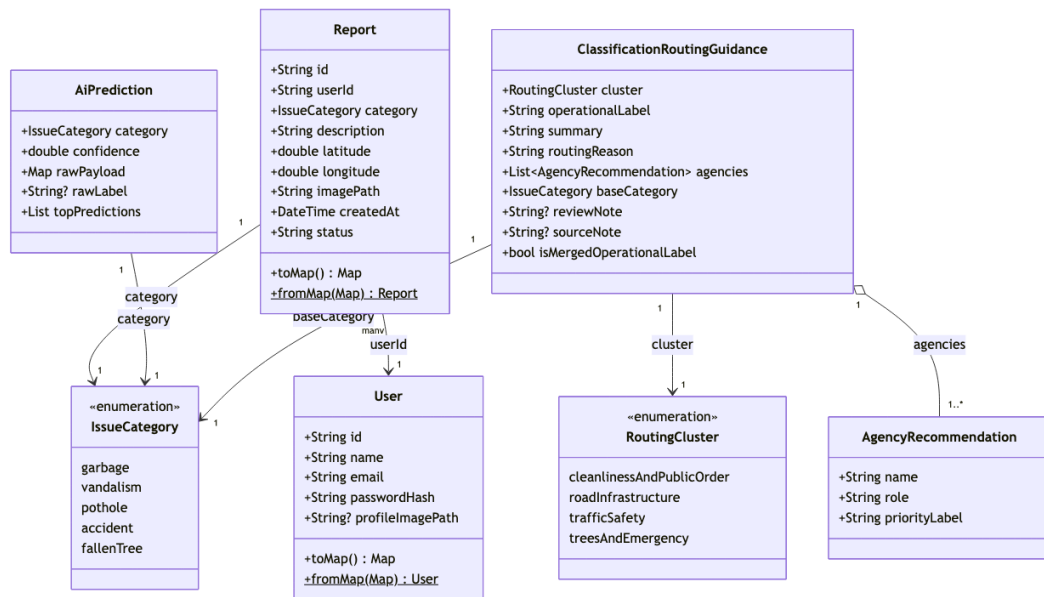
3.4.4 Model Domain (Diagram Kelas UML)

Gambar 3.7 mengilustrasikan struktur kelas domain utama dalam aplikasi.

Kelas-kelas domain utama adalah:

- **Report:** Entitas utama yang merepresentasikan satu laporan. Atribut: `id`, `userId`, `category (IssueCategory)`, `description`, `latitude`, `longitude`, `imagePath`, `createdAt`, `status`.

Diagram Kelas — Domain Model Aplikasi

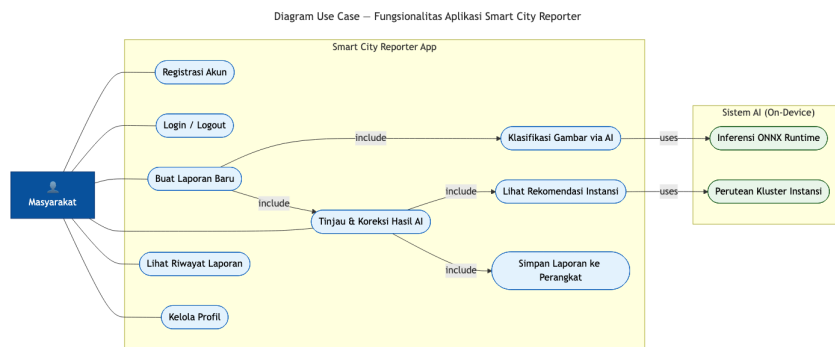


Gambar 3.7. Diagram Kelas — Domain Model Aplikasi

- **User:** Entitas pengguna untuk autentikasi lokal. Atribut: `id`, `name`, `email`, `passwordHash`, `profileImagePath`.
- **AiPrediction:** Hasil inferensi model ONNX. Atribut: `category` (`IssueCategory`), `confidence` (double 0–1), `rawPayload` (Map berisi label mentah dan top predictions).
- **IssueCategory:** Enum yang merepresentasikan lima kategori semantik aplikasi: `garbage`, `vandalism`, `pothole`, `accident`, `fallenTree`. Label dataset mentah (7 kelas) dipetakan ke enum ini oleh metadata model.
- **ClassificationRoutingGuidance:** Hasil mesin perutean. Atribut: `cluster` (`RoutingCluster`), `operationalLabel`, `summary`, `routingReason`, `agencies` (`List<AgencyRecommendation>`), `reviewNote`.
- **AgencyRecommendation:** Rekomendasi instansi individual. Atribut: `name`, `role`, `priorityLabel`.

3.4.5 Kasus Penggunaan (Diagram *Use Case* UML)

Gambar 3.8 merangkum seluruh fungsionalitas sistem yang dapat diakses oleh pengguna.



Gambar 3.8. Diagram *Use Case* — Fungsionalitas Aplikasi

Kasus penggunaan utama yang tersedia bagi **Masyarakat** adalah:

1. **Registrasi dan Login:** Autentikasi berbasis penyimpanan lokal (tanpa server).
2. **Buat Laporan Baru:** Mengambil foto, mengisi deskripsi opsional, mengaktifkan klasifikasi AI.
3. **Tinjau Hasil Klasifikasi AI:** Melihat kategori prediksi, *confidence*, dan koreksi jika diperlukan.
4. **Lihat Rekomendasi Instansi:** Melihat instansi yang direkomendasikan beserta alasan perutean.
5. **Simpan Laporan:** Menyimpan laporan dengan data foto, GPS, kategori, dan metadata ke SQLite.
6. **Lihat Riwayat Laporan:** Menelusuri laporan-laporan yang telah dibuat sebelumnya.
7. **Kelola Profil:** Memperbarui informasi profil pengguna.

3.5 Mekanisme Perutean Instansi

Modul `classification_routing.dart` mengimplementasikan mekanisme perutean berbasis aturan (*rule-based routing*) yang memetakan hasil klasifikasi AI ke rekomendasi instansi pemerintah daerah. Mekanisme ini beroperasi dalam dua tahap:

Tahap 1 — Normalisasi Label: Label mentah (*raw label*) dari model ONNX disesuaikan dengan kasus-kasus khusus yang memerlukan penanganan berbeda dari pemetaan kategori standar. Misalnya, label `Accident Issues` secara eksplisit diarahkan ke kluster keselamatan lalu lintas dengan layanan darurat sebagai prioritas utama, terlepas dari kategori `IssueCategory` yang dipetakan secara otomatis.

Tahap 2 — Pemetaan ke Kluster: Setiap laporan diarahkan ke salah satu dari empat kluster operasional, sebagaimana disajikan pada Tabel 3.4.

Tabel 3.4. Kluster Operasional Perutean Instansi

Kluster	Label Operasional	Instansi Utama
<i>Cleanliness & Public Order</i>	Kebersihan & Ketertiban Lingkungan	Dinas Lingkungan Hidup (utama) + Satpol PP (pendukung)
<i>Road Infrastructure</i>	Jalan & Prasarana Umum	Dinas Bina Marga / PUPR (utama) + Dishub (pendukung)
<i>Traffic Safety</i>	Keselamatan Lalu Lintas & Respons Darurat	PSC 119 + Kepolisian 110 (darurat) + Dishub (pendukung)
<i>Trees & Emergency</i>	Pohon Tumbang & Respons Cepat	BPBD (utama) + Dinas Pertamanan (pendukung)

Untuk prediksi dengan label `Mixed Issues` (yang tidak dikecualikan dari model dalam *deployment* aktual), sistem menggunakan prediksi kelas tertinggi kedua (*secondary top prediction*) sebagai dasar perutean, disertai catatan peninjauan manual (*review note*) kepada pengguna.

Seluruh logika perutean bersifat deterministik dan beroperasi sepenuhnya *in-memory* tanpa akses jaringan, memastikan rekomendasi tersedia secara instan bahkan dalam kondisi tanpa koneksi internet.

3.6 Struktur Modul Aplikasi Flutter

Aplikasi Flutter menggunakan arsitektur berbasis fitur (*feature-based architecture*) di bawah direktori `lib/`. Setiap fitur mengenkapsulasi layar, *provider* Riverpod, dan model yang relevan. Tabel 3.5 merangkum modul fitur utama.

Tabel 3.5. Modul Fitur Aplikasi Flutter

Modul Fitur	Fungsi Utama
<code>auth</code>	Registrasi, login, dan manajemen sesi lokal
<code>splash</code>	Layar pembuka dan inisialisasi aplikasi
<code>onboarding</code>	Panduan pengguna baru
<code>dashboard</code>	Ringkasan laporan dan navigasi utama
<code>reports</code>	Pembuatan laporan, klasifikasi AI, dan riwayat
<code>profile</code>	Manajemen profil pengguna
<code>settings</code>	Pengaturan aplikasi
<code>testing</code>	Pengujian inferensi model ONNX secara mandiri

Modul `reports` merupakan fitur inti yang mengintegrasikan seluruh komponen utama sistem: kamera, GPS, inferensi ONNX, perutean instansi, dan penyimpanan data lokal. Alur kerja dalam modul ini terdiri dari tiga layar berurutan: `CreateReportScreen` → `AiResultReviewScreen` → konfirmasi dan penyimpanan.

DAFTAR PUSTAKA

- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.
- Fang, Y., Sun, Q., Wang, X., Huang, T., Wang, X., and Cao, Y. (2023). EVA-02: A Visual Representation Powerhouse Unleashed by CLIP and MAE.
- Firgia, L., Muhamad Muslih, and Aditya Pratama (2022). Implementasi Sistem Informasi Pengaduan Masyarakat Di Daerah Perbatasan Studi Kasus Desa Cipta Karya. *Jurnal Rekayasa Teknologi Nusa Putra*, 8(2):101–110.
- Google LLC (2024). Flutter documentation. <https://flutter.dev/docs>. Accessed: 2024.
- Liu, X., Peng, H., Zheng, N., Yang, Y., Hu, H., and Yuan, Y. (2023a). EfficientViT: Memory Efficient Vision Transformer with Cascaded Group Attention.
- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. (2023b). ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders.
- Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization.
- Microsoft (2024). ONNX Runtime: Cross-platform, high performance ML inferencing. <https://github.com/microsoft/onnxruntime>. Version 1.18. GitHub repository.
- Rousselet, R. (2024). Riverpod: A reactive caching and data-binding framework for Flutter. <https://riverpod.dev>. Accessed: 2024.

- Ryali, C., Hu, Y.-T., Bolya, D., Wei, C., Fan, H., Huang, P.-Y., Aggarwal, V., Chowdhury, A., Poole, O., Dollar, P., Girshick, R., and Feichtenhofer, C. (2023). Hiera: A Hierarchical Vision Transformer without the Bells-and-Whistles.
- Sadiq, T. and Omlin, C. W. (2025). Sensing in Smart Cities: A Multimodal Machine Learning Perspective.
- Shen, J. and Hu, C. (2025). Smart City Governance Based on Multimodal Large-Scale Models. In Guo, W., Qian, K., Xu, W., and Wang, P., editors, *Proceedings of the 3rd International Conference on Green Building, Civil Engineering and Smart City (GBCESC 2024)*, volume 264, pages 915–924. Atlantis Press International BV, Dordrecht.
- Siret, I. and Sabadie, W. (2022). Public complaining: A blessing in disguise? Educational calling as a benevolent process that gives consumers voice on brands' social media. *Journal of Business Research*, 150:476–490.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. (2016). Rethinking the Inception Architecture for Computer Vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826. IEEE.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention Is All You Need.
- Yu, W. and Wang, X. (2024). MambaOut: Do We Really Need Mamba for Vision?